Problem Statement: Centralized Log Aggregation & Crash Analytics Platform

The Objective

Modern distributed systems generate massive volumes of system logs and performance metrics. When a critical failure occurs, engineering teams need immediate, centralized visibility into the application's state.

Your challenge for this 36-hour hackathon is to architect and build a high-throughput, centralized log aggregation and crash-tracking platform (similar to Datadog or Sentry). You will build the ingestion backend, a role-based developer portal, and the asynchronous client SDKs required to capture application logs and crash data from external applications.

---

1. Multi-Tenant Role-Based Access Control (RBAC)

The system must support a strict hierarchical access model. The database schema must correctly handle the complex relational mapping between Users, Teams, Projects, and Sub-modules.

- **Global Admin:** Full read-access to all logs and activity reports across the entire organization.
- **Project Managers:** Assigned to specific *Projects*. They have read-access to all logs generated within their Project and can view the Teams assigned to it.
- **Developers & Teams (Many-to-Many):** A developer can belong to multiple Teams. Teams are assigned to specific *Sub-modules* within a Project.
- **Data Isolation:** When a developer logs into the portal, queries must be strictly scoped. They must **only** see logs and stack traces originating from the exact Sub-modules their Team is assigned to.

2. The Ingestion Engine & Backend Architecture

The backend must be designed to handle high-throughput log ingestion without dropping payloads or crashing.

- **Standard Logging:** Categorize logs by standard severities: INFO, WARN, ERROR, and FATAL.
- **Payload Structure:** Ingested payloads must include metadata (timestamp, environment, module ID) and the raw message.
- **Stack Trace Parsing:** For ERROR and FATAL severities, the backend must accept and securely store multi-line stack traces.
- **Issue Lifecycle:** On the frontend, ERROR and FATAL logs must be groupable into "Issues." Managers can assign an Issue to a developer, who can then mutate its state to RESOLVED.
- **Database Optimization (Crucial):** Relational databases degrade during heavy LIKE %...% text searches. You must optimize your PostgreSQL architecture for rapid log querying. *Hint: Participants are expected to implement PostgreSQL Full-Text Search, GIN (Generalized Inverted Index) indexing, or table partitioning to ensure search latency remains low as the dataset grows.*

---

3. Asynchronous Client SDKs

External applications will integrate your SDKs to transmit data. **A poorly written SDK will crash the host application.** Your SDKs must be highly performant and network-resilient.

- **Mandatory SDKs:** Java and C++ (Bonus for JS, Go, Kotlin, or Python).
- **Non-Blocking I/O:** The SDK **must not** block the host application's main execution thread. Network calls to your ingestion API must be handled asynchronously via background threads or worker pools.
- **Batching:** Instead of making an HTTP request for every single INFO log, the SDK should ideally batch logs and flush them periodically or when a threshold is reached.
- **True Crash Reporting:** Capturing a standard exception is not enough. Your SDK must implement global exception handlers (e.g., Thread.setDefaultUncaughtExceptionHandler in Java) to capture and transmit FATAL crashes right before the host application process dies.

---

4. Strict Technology Stack

- **Backend Ingestion API:** Spring Boot (Java)
- **Datastore:** PostgreSQL (Optimized for heavy text-search workloads)
- **Frontend Portal:** React.js paired with Shadcn UI for a responsive, data-dense dashboard.

---

5. What to Build?

The Project: Develop a centralized log aggregation system that captures and categorizes error reports from distributed applications.

- **Core Task:** Construct a centralized logging pipeline for applications to submit data. You must also integrate Role-Based Access Control (RBAC) to ensure that logs are accessible only to database administrators and authorized personnel.

Judging Criteria: Teams will be evaluated heavily on the architectural soundness of their backend ingestion, the multi-threading implementation in their SDKs, and the efficiency of their database querying.

(:-Happy Hacking-:)